



TILERA[®]

APPLICATION NOTE:

MANAGING TILE-GX Host PCI EXPRESS PACKET QUEUES

RELEASE 1.20
DOC. NO. AN044
APRIL 2013
TILERA CORPORATION

Copyright © 2008-2013 Tiler Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tiler Corporation: Tiler and the Tiler logo.

The following are trademarks of Tiler Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, TILEExpressPro-20G, TILEExpressPro-22G, iMesh, TileDirect, TILExtreme-Gx, TILExtreme-Gx Duo, TILEmpower, TILEmpower-Gx, TILEncore, TILEncorePro, TILEncore-Gx, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx72, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, TILE-Gx8009, TILE-Gx8016, TILE-Gx8036, TILE-Gx3036, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, iLib, TMC (Tiler Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tiler IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tiler products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tiler Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tiler disclaims any express or implied warranty relating to the sale and/or use of Tiler products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. Tiler assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number	AN044
Release	1.20
Date	4/10/13

Contact Information:

Tiler Corporation
Information info@tilera.com Web Site http://www.tilera.com

CONTENTS

1 INTRODUCTION	1
2 INTENDED AUDIENCE	2
3 DEVELOPMENT ENVIRONMENT REQUIREMENTS	2
4 DESCRIPTION OF THE APPLICATION	2
5 SOFTWARE ARCHITECTURE OF THE HOST PCIe PACKET QUEUE APPLICATION ...	2
5.1 Host-side API	2
5.1.1 Creating a PCIe Communications Endpoint	4
5.1.2 Configuring the Host Ring Buffer Size	4
5.1.3 Mapping the Host Ring Buffer to User Space	6
5.1.4 Mapping Host Buffer Ring Index Registers to User Space	6
5.1.5 Mapping the Packet Queue Channel Status Register to User Space	7
5.1.6 Conducting Packet Transfers To and From the Tile	7
5.2 Tile-Side API	7
5.2.1 Creating a GXIO TRIO and a GXPCI Context	8
5.2.2 Opening a Packet Queue Interface	8
5.2.3 Allocating and Registering the Data Buffer Pool	9
5.2.4 Conducting Packet Transfer To and From the Host	9
6 CONFIGURING AND RUNNING THE APPLICATION AND TEST EQUIPMENT	10
6.1 Running Multiple Packet Queue Tests	10
6.2 Running Duplex Tests in Separate TILE-Gx Processes	11
6.3 Running Duplex Tests in a Single TILE-Gx Process	11
7 CONFIGURING DIFFERENT OPTIONS	11
8 IMPROVING PERFORMANCE ON A TILE-Gx PROCESSOR	11
9 PERFORMANCE MEASUREMENTS AND RESULTS	12
9.1 Performance Data for 64-Byte Packets	12

CONTENTS

9.2 Performance Data for 128-Byte Packets	12
9.3 Performance Data for 256-Byte Packets	13
9.4 Performance Data for 4096-byte Packets	13
10 POTENTIAL AREAS FOR ADDITIONAL IMPROVEMENTS	13
11 SUMMARY	13

MANAGING TILE-Gx Host PCI EXPRESS PACKET QUEUES

1 Introduction

Traditionally, software applications that transfer data between the Tile™ platform and the host over a PCIe link use either:

- A socket-like character streams API
- A zero-copy command queue API

Both of these APIs are tailored for high bandwidth, large-data-buffer transfers. These methods also rely heavily on Linux system calls and interrupts in the data path, which is inefficient for servicing small data buffers (≤ 1024 bytes).

The vastly improved PCIe design, available with the TILE-Gx™ processor family, allows you to use high-performance small data buffer transfer APIs for host communication. The Host PCIe Packet Queue API implements highly-efficient packet transmission data paths that provide extremely low latency and close to line-rate for moving data buffers between the Tile system and the host, without involving OS system calls and interrupts in the data transfer path.

This application note explains Host PCIe Packet Queue API and demonstrates how applications can use that API to implement a high-performance PCIe data transmission framework.

2 Intended Audience

This document is intended for use by:

- Software developers using packet queues in the Multicore Development Environment™ on TILE-Gx devices and platforms
- Field Application Engineers

3 Development Environment Requirements

To run this code, you need a TILE-Gx-based TILEncore-Gx™ PCIe card installed in an x86 Linux host platform, and any of the Tiler MDE 4.x releases.

4 Description of the Application

You can build any applications that require low-latency and high throughput data transfers between the Tile and the host systems on top of the Packet Queue API. Two typical applications are:

- Packet capturing
- Host off-loading

For Packet capture applications, ethernet packets arrive at the Tile XAUI interfaces and after some pre-processing on the Tile processor are transferred to the host where they are processed and written to some persistent storage devices.

A typical host off-loading application involves transferring data from the host to the Tile system which performs efficient computing operations on the data and returns the processed data back to the host memory. This process takes advantage of the TILE-Gx processor's strong capability in performing tasks such as video encoding/decoding, encryption/decryption and compression/decompression, etc. In essence, the TILE-Gx CPU serves as the dataplane for the x86 control plane.

5 Software Architecture of the Host PCIe Packet Queue Application

The Packet Queue API provides support for direct data transfers between the Tile user space and the host user space without passing the data through the OS kernel space on either sides of the channel.

5.1 Host-side API

To achieve the high throughput and low latency goals for the Packet Queue API, the host API is designed on the same principle as the tile side: avoiding system calls and interrupts by mapping the host packet ring buffer into user space. The host application can directly operate on the packet buffers without having to copy the data payload between the kernel space and the user space.

The host packet ring buffer is allocated by the packet queue component of the host PCIe driver (hereafter referred to as Packet Queue driver) and mapped into user space. It is organized as a circular ring of buffers of equal sizes, managed through two index registers, the producer index and the consumer index, which are implemented using unsigned integer arithmetic on the tile side as part of the PCI CSR (control and status registers). The two index registers are mapped to the host's user space to enable the host application to acquire the incoming packet efficiently and inform the tile application of the host buffer availability for flow control purpose.

In the case of a tile-to-host transfer, the producer index points to the next empty buffer that will be written by the Tile application. The Tile application transfers a packet to the empty buffer pointed to by the producer index and advances the producer index, which is read-only for the host application. The consumer index points to the next packet that the host application can process. All the packet buffers from the consumer index up to, but not including, the producer index contain valid packets. When the consumer index equals the producer index, the ring is empty. The host application writes the consumer index with the index just after the last packet it has processed to free those packet buffers. The tile application uses the producer index and the consumer index register to determine the host buffer availability. Therefore, the host application needs to update the consumer index register periodically. Since the consumer index update is done by a PCIe memory write operation to the tile application, the update interval should be as long as possible without causing the tile application to stop sending packets. Similarly, every read of the producer index register is a PCIe memory read operation. The host application should read the producer index once and process as many packets as the producer index indicates are available before doing another read of the producer index. In the case of host-to-tile transfer, the producer index is updated by the host application after a packet is deposited into the host ring buffer by the host application and the consumed index is updated by the tile application after the packet is consumed by the tile application.

Both the producer and consumer indexes are 32-bit absolute indexes. The relative indexes into the ring buffer are obtained on the fly by division, modulo the host ring buffer's number of entries.

The host-side API provides functions for doing the following:

- Creating a PCIe communications endpoint. See [“Creating a PCIe Communications Endpoint” on page 4](#).
- Configuring the host ring buffer size. See [“Configuring the Host Ring Buffer Size” on page 4](#).
- Mapping the host ring buffer to user space. See [“Mapping the Host Ring Buffer to User Space” on page 6](#).
- Mapping the host buffer ring producer and consumer index registers to user space. See [“Mapping Host Buffer Ring Index Registers to User Space” on page 6](#).
- Mapping the channel status register for the Packet Queue to user space. See [“Mapping the Packet Queue Channel Status Register to User Space” on page 7](#).

- Conducting packet transfer to and from the Tile. See [“Conducting Packet Transfers To and From the Tile”](#) on page 7.

5.1.1 Creating a PCIe Communications Endpoint

You can create a PCIe communication endpoint on the host side that serves as the file handle for the physical PCIe link. Do this by opening the special device file representing the packet queue device, named `/dev/tilegxpciN/packet_queue/h2t/M`, where *N* identifies which TILEncore-Gx cards you have installed in the x86 Linux host machine and *M* is packet queue index in the range of `[0, GXPCI_HOST_PQ_T2H_COUNT - 1]`. Strings *t2h* and *T2H* stand for *Tile-to-Host*, indicating the data transfer direction. Similarly, *h2t* and *H2T* stand for *Host-to-Tile*.

For example, from the example file `$TILER_ROOT/examples/trio/gxpci_host_pq/host.c`, this code creates the PCIe communications endpoint:

```
char dev_name[40];
snprintf(dev_name,
         sizeof(dev_name),
         "/dev/tilegxpci0/packet_queue/h2t/%d",
         queue_index);
int pq_fd = open(dev_name, O_RDWR);
if (pq_fd < 0)
{
    fprintf(stderr,
           "Host: Failed to open '%s': %s\n",
           dev_name,
           strerror(errno));
    abort();
}
```

The open attempt fails if the Tile processor has not been booted or if the tile side application has not opened its Packet Queue device within a 10-second time-out period.

Note: On host systems that suffer from fragmented system memory, e.g. after running for a while, the `open()` could fail because it cannot allocate the physically contiguous host ring buffer. Rebooting the host fixes this problem.

5.1.2 Configuring the Host Ring Buffer Size

The host ring buffer is composed of one or more discreet segments. Each segment is a chunk of physically contiguous memory that is divided into individual packet buffers of equal sizes. By default, the host ring buffer has a single segment of size 4MB, consisting of 1024 entries of 4KB packet buffers. The maximum segment size is determined by the size of maximum physically contiguous memory that can be allocated on the host system. On a generic x86 Linux host system, 4MB is the maximum physically contiguous buffer that can be allocated, without reconfiguring the host kernel.

To request a host ring buffer size larger than 4MB, use the x86 Linux host driver parameters `pq_t2h_mems=` and `pq_h2t_mems=` to specify the total ring buffer sizes of the tile-to-host and host-to-tile packet queue interfaces, respectively.

The parameters take the following format:

```
[size] [, numa_node] [, share_ring] [: [size] [, numa_node]] [, share_ring]
```

Each parameter triplet specifies the following:

size: Total ring buffer size, with a default of 4MB. You can set sizes using a suffix of K for kilobytes, M for megabytes and G for gigabytes.

numa_node: NUMA zone of the buffer, with a default of 0.

share_ring: If the ring buffer is shared between the T2H and H2T interfaces with the same interface number, with 1 indicating a shared buffer. By default, this parameter is 0 since each unidirectional interface has its own ring buffer. Using a shared ring buffer is preferred in loopback-like applications because such a shared buffer can avoid the expensive memory copy operation when queuing an ingress (T2H) packet to the egress (H2T) queue.

The relative position of the triplet in the parameter string indicates the Packet Queue interface number.

The following is an example of a host driver installation command setting the buffers to 2MB using shared ring buffers:

```
tilepci-install pq_t2h_mems=2M,0,1 pq_h2t_mems=2M,0,1
```

Note: The triplet setting for the H2T/T2H queues with the same interface number must be identical.

You define the maximum number of segments in a host ring buffer using the macro `HOST_PQ_SEGMENT_MAX_NUM`. The maximum allowed value is 32, which means that you can request up to 128MB for the ring buffers. Use the macro `GXPCI_HOST_PQ_SEGMENT_ENTRIES` to define the number of buffer entries in a segment.

Note: The value of `GXPCI_HOST_PQ_SEGMENT_ENTRIES` is set to 1024 by default. We strongly suggest that you do not change this value. A 128MB ring buffer provides 32*1024 packet buffers.

You can set the individual packet buffer size to any value that is smaller than 4096-byte and is a multiple of 2. For example:

```
tilepci_packet_queue_info_t buf_info;
buf_info.buf_size = host_buf_size;          /* 1024, 512, etc. */
int err = ioctl(pq_fd, TILEPCI_IOC_SET_PACKET_QUEUE_BUF, &buf_info);
if (err < 0)
{
    fprintf(stderr, "Host: Failed TILEPCI_IOC_SET_PACKET_QUEUE_BUF: %s\n",
        strerror(errno));
    abort();
}
```

5.1.3 Mapping the Host Ring Buffer to User Space

To achieve the highest possible data transfer performance, the Packet Queue driver maps the host ring buffer to the user space so that the host application can access the PCI DMA buffer directly, by means of the `mmap()` system call starting at the `mmap` file offset `TILEPCI_PACKET_QUEUE_BUF_MMAP_OFFSET`.

For example:

```
void* buffer =
    mmap(0,
        host_ring_buffer_size,
        PROT_READ | PROT_WRITE,
        MAP_SHARED,
        pq_fd,
        TILEPCI_PACKET_QUEUE_BUF_MMAP_OFFSET);
assert(buffer != MAP_FAILED);
```

5.1.4 Mapping Host Buffer Ring Index Registers to User Space

The host buffer ring's producer and consumer index participate in the flow-control process between the Tile and the Gx applications. The host application relies on these two registers to decide when and where to start and consume the packet transfer. The host application gains direct access to these two registers by means of the `mmap()` system call starting at the `mmap` file offset `TILEPCI_PACKET_QUEUE_INDICES_MMAP_OFFSET`.

For example:

```
struct gxpci_host_pq_regs_app* pq_regs =
    (struct gxpci_host_pq_regs_app*)
    mmap(0,
        sizeof(struct gxpci_host_pq_regs_app),
        PROT_READ | PROT_WRITE,
        MAP_SHARED,
        pq_fd,
        TILEPCI_PACKET_QUEUE_INDICES_MMAP_OFFSET);
assert(pq_regs != MAP_FAILED);
```

5.1.5 Mapping the Packet Queue Channel Status Register to User Space

When either side of the applications exits or crashes, the other side must be informed of the event and the channel must be reset gracefully so that it can be re-opened subsequently. The Packet Queue driver is responsible for managing the Packet Queue channel state machine and mapping the channel status to the user space via the `mmap()` system call starting at the `mmap` file offset `TILEPCI_PACKET_QUEUE_STS_MMAP_OFFSET`.

For example:

```
struct tlr_pq_status *pq_status=
    mmap(0,
        sizeof(struct tlr_pq_status),
        PROT_READ | PROT_WRITE,
        MAP_SHARED,
        pq_fd,
        TILEPCI_PACKET_QUEUE_STS_MMAP_OFFSET);
assert(pq_status != MAP_FAILED);
```

Once the Packet Queue channel status register is mapped to the user space, the application can monitor the channel status by polling the status register as follows:

```
if (pq_status->status == GXPCI_CHAN_RESET)
{
    printf("Host: H2T: channel is reset\n");
    goto done;
}
```

5.1.6 Conducting Packet Transfers To and From the Tile

Now that all the preparatory work is done, the host application can start the loop of packet exchanges with the corresponding tile application, by polling the host buffer ring's index registers and the channel status register. For more details, refer to the sample host application in `$TILER_ROOT/examples/trio/gxpci_host_pq/host.c`.

5.2 Tile-Side API

The tile-side Packet Queue API is part of the GXPCI API, outlined in the *Application Libraries Reference Manual* (UG527).

It provides functions for doing the following:

- Creating a GXIO TRIO and a GXPCI context. See [“Creating a GXIO TRIO and a GXPCI Context” on page 8](#).
- Opening a packet queue interface. See [“Opening a Packet Queue Interface” on page 8](#).
- Allocating and registering the data buffer pool. See [“Allocating and Registering the Data Buffer Pool” on page 9](#).
- Conducting packet transfer to and from the host. See [“Conducting Packet Transfer To and From the Host” on page 9](#).

5.2.1 Creating a GXIO TRIO and a GXPCI Context

The GXIO TRIO and GXPCI contexts provide abstraction and encapsulation for the data transfer engine implementation utilizing the TILE-Gx PCIe hardware. The user application provides the storage or memory for these two contexts and treats them as opaque objects.

For example:

```
gxio_trio_context_t trio_context_body;
gxio_trio_context_t* trio_context = &trio_context_body;
gxpci_context_t gxpci_context_body;
gxpci_context_t* gxpci_context = &gxpci_context_body;

result = gxio_trio_init(trio_context, trio_index);
result = gxpci_init(trio_context, gxpci_context, trio_index, loc_mac);
```

For detailed API function descriptions, refer to the *Application Libraries Reference Manual* (UG527).

5.2.2 Opening a Packet Queue Interface

The application must open a Packet Queue interface or queue as transport for packet transfers. For example, to open a tile-to-host queue with `queue_index` which matches the `queue_index` that is used by the host application in its `open()` system call, invoke the following function:

```
int asid = GXIO_ASID_NULL;
result = gxpci_open_queue(gxpci_context,
                        asid,
                        GXPCI_PQ_T2H,
                        0, queue_index,
                        0,
                        0);
```

The argument `asid` indicates the Address Space ID (ASID), a term that is borrowed from the OS. The TILE-Gx PCIe hardware uses the ASID to set up the DMA access to the application memory space from the TILE-Gx endpoint controller. For a stand-alone Packet Queue application, `GXIO_ASID_NULL` is used as a hint to the API that a new ASID should be allocated inside the API. In the common case of a joint mPIPE™ (multicore Programmable Intelligent Packet Engine) and PCIe application, you can explicitly allocate a single ASID using the GXIO routine `gxio_trio_alloc_asids()` routine and then use the shared ASID in both parts of the application.

The `gxpci_open_queue()` routine blocks until the host application calls the `ioctl()` routine with the command `TILEPCI_IOC_SET_PACKET_QUEUE_BUF`.

5.2.3 Allocating and Registering the Data Buffer Pool

Packet buffers that are accessed directly by the PCIe hardware must be registered with the hardware which creates I/O TLB entries to facilitate zero-copy data transfer between the TILE-Gx memory and the PCIe link. The following shows how to allocate a huge page and register it with the hardware:

```
tmc_alloc_t alloc = TMC_ALLOC_INIT;
tmc_alloc_set_huge(&alloc);
void* buf_mem = tmc_alloc_map(&alloc, MAP_LENGTH);
assert(buf_mem);

result = gxpci_iomem_register(gxpci_context, buf_mem, MAP_LENGTH);
```

Note: You must call the `gxpci_iomem_register()` routine after you invoke the `gxpci_open_queue()` routine.

5.2.4 Conducting Packet Transfer To and From the Host

Similar to the host application, the tile application should run in a loop. Every time the application enters the loop, it should perform the following operations:

1. Determine the number of available command credit which is the maximum number of send or receive buffers that can be posted to the Packet Queue. Your application needs to monitor if the channel has been reset by the host application by checking for the `GXPCI_ERESET` value. For example:

```
while (!done)
{
    int credits = gxpci_get_cmd_credits(context);
    if (credits == GXPCI_ERESET)
    {
        printf("Tile: do_send: channel is reset %d\n", seq_num);
        done=1;
        break;
    }
}
```

2. Post as many commands as possible by invoking functions `gxpci_pq_t2h_cmd()` or `gxpci_pq_h2t_cmd()`.

For example:

```
unsigned count = MIN(MAX_CMDS_BATCH, credits);

// Post the next 'count' buffers.
for (int i = 0; i < count; i++)
{
    cmd[i].buffer = buf_mem + (
        (seq_num << pkt_size_order)
        & (MAP_LENGTH - 1));

    for (j=0; j<pkt_size; j+=4)
    {
        *((uint32_t *) (cmd[i].buffer + j)) = data_pattern++;
    }
}
```

```
// make data visible to push DMA command
__insn_mf();

result = gxpci_pq_t2h_cmd(gxpci_context, &cmd[i]);
seq_num++;
}
```

3. Check if there are any completed commands by invoking the `gxpci_get_comps()` routine. Since this function could block, you should always check for `GXPCI_ERESET` which is returned when the function is unblocked due to a channel reset operation. For example:

```
result = gxpci_get_comps(gxpci_context, comp, 0, MAX_CMDS_BATCH);
if (result == GXPCI_ERESET)
{
    printf("Tile: do_send %d: channel is reset\n");
    done = 1;
    break;
}
```

Note: If you want to use any non-zero value for the minimum number of completions in the `gxpci_get_comps()` routine (passing a non-zero value to the 3rd parameter), the host application must update the flow-control registers after producing or consuming every packet. In the example host application program (`host.c`), disable the `TAIL_UPDATE_LIMIT_ENABLE` macro.

6 Configuring and Running the Application and Test Equipment

Refer to the `README` file in `$TILERA_ROOT/examples/trio/gxpci_host_pq` for steps to run example applications, which generates performance numbers such as those listed in [“Performance Measurements and Results” on page 12](#).

6.1 Running Multiple Packet Queue Tests

To run multiple pairs of applications to exercise multiple Packet Queue interfaces, use commands like the following:

On host side:

```
/tmp/host_t2h & /tmp/host_t2h --queue_index=1 &
/tmp/host_t2h -- queue_index=2 &
/tmp/host_t2h --queue_index=3
```

On tile side using the `cpu_rank` to specify the worker tile:

```
/tile_t2h & /tile_t2h -- queue_index=1 --cpu_rank=2 &
/tile_t2h --queue_index=2 --cpu_rank=3 &
/tile_t2h --queue_index=3 --cpu_rank=4
```

For example, assuming that you have booted the TILEncore-Gx PCIe card and that you have uploaded the `tile_t2h` and `tile_h2t` programs, run this command:

```
$ tile-monitor --dev gxpci0 --resume --run \
- sh -c "/tile_t2h & /tile_t2h --queue_index=1 --cpu_rank=2 & \
/tile_t2h --queue_index=2 --cpu_rank=3 & \
/tile_t2h --queue_index=3 --cpu_rank=4" - --quit
```

6.2 Running Duplex Tests in Separate TILE-Gx Processes

The Packet Queue interfaces are designed to be independent and unidirectional. Each unidirectional interface is usually managed by a unique process. To run duplex tests, start two Tile processes as follows:

On the host side:

```
/tmp/host_t2h & /tmp/host_h2t
```

On the tile side:

```
/tile_t2h & /tile_h2t
```

6.3 Running Duplex Tests in a Single TILE-Gx Process

On the Tile side, you can execute duplex data transfers by running a single Tile process which spawns separate threads for unidirectional queues. This arrangement is actually mandatory if you need eight pairs of T2H and H2T interfaces. To run duplex tests in a single Tile process, do something like the following:

On the host side:

```
/tmp/host_t2h & /tmp/host_h2t
```

On the tile side:

```
/duplex
```

where the duplex Tile executable is built from the example program `duplex.c`.

7 Configuring Different Options

By default, the Packet Queue API configures eight T2H interfaces and eight H2T interfaces, as specified by `GXPCI_HOST_PQ_T2H_COUNT` and `GXPCI_HOST_PQ_H2T_COUNT` host driver macros.

8 Improving Performance on a TILE-Gx Processor

To achieve the maximum data transfer performance, use a packet size that is a multiple of the cache line size on a x86 Linux host, i.e. 64 bytes. Otherwise, the read-modify-write cache operations present significant overhead to data transfer efficiency for the host machine.

9 Performance Measurements and Results

This section presents some performance measurement results from the example program over a PCIe Gen2 x8 link between a TILeCore-Gx PCIe card with the Tile-Gx36 running at 1GHz and a 4-CPU host with Intel Xeon™ CPU @ 3.20GHz, demonstrating excellent performance and scalability.

9.1 Performance Data for 64-Byte Packets

Using a 64-byte packet size, we produced the following results for Tile-to-host, Host-to-Tile and Duplex data transfers:

Packet Size	Number of Queues	Transfer Direction	Throughput (Gbps)	Packet Rate (Mpps)
64 bytes	1	T2H	8.08	15.78
64 bytes	1	H2T	10.26	20.03
64 bytes	4	T2H	22.25	43.44
64 bytes	4	H2T	19.93	38.93
64 bytes	1xT2H,1xH2T	Duplex	8.12,10.24	15.85,20.00
64 bytes	4xT2H,4xH2T	Duplex	16.19, 17.75	31.61, 34.67

9.2 Performance Data for 128-Byte Packets

Using a 128-byte packet size, we produced the following results for Tile-to-host, Host-to-Tile and Duplex data transfers:

Packet Size	Number of Queues	Transfer Direction	Throughput (Gbps)	Packet Rate (Mpps)
128 bytes	1	T2H	16.28	15.90
128 bytes	1	H2T	20.46	19.98
128 bytes	2	T2H	26.2	25.59
128 bytes	2	H2T	24.59	24.01
128 bytes	3	T2H	25.95	25.43
128 bytes	3	H2T	24.17	23.61
128 bytes	1xT2H,1xH2T	Duplex	16.21,20.49	15.83,20.01

9.3 Performance Data for 256-Byte Packets

Using a 256-byte packet size, we were able to saturate the PCIe link using a single packet queue, and produced the following results for Tile-to-host, Host-to-Tile and Duplex data transfers:

Packet Size	Number of Queues	Transfer Direction	Throughput (Gbps)	Packet Rate (Mpps)
256 bytes	1	T2H	28.75	14.04
256 bytes	1	H2T	27.58	13.47
256 bytes	1xT2H,1xH2T	Duplex	26.30,27.19	12.84,13.28

9.4 Performance Data for 4096-byte Packets

Using a 4096-byte packet size, we were able to saturate the PCIe link using a single packet queue, and produced the following results for Tile-to-host, Host-to-Tile and Duplex data transfers:

Packet Size	Number of Queues	Transfer Direction	Throughput (Gbps)	Packet Rate (Mpps)
4096 bytes	1	T2H	28.75	0.88
4096 bytes	1	H2T	27.75	10.85
4096 bytes	1xT2H,1xH2T	Duplex	28.41,27.44	0.87,0.84

10 Potential Areas for Additional Improvements

You can enhance the Packet Queue API to support more Packet Queue interfaces.

11 Summary

This application note describes the Packet Queue API that enables applications that require high-speed data buffer transfers between the Tile and the host with extremely low latency. The performance comes very close to the maximum possible transfer rate over the PCIe links and scales well with the number of data transfer channels.

Summary
